

LAPORAN PROYEK
MATA KULIAH ARTIFICIAL INTELLIGENCE

**ANALISIS DAN IMPLEMENTASI
DYNAMIC A* WITH SCENARIO ENGINE**

Sistem Navigasi Jalur Terpendek Adaptif untuk Kampus UNIB



Disusun oleh:

Ali Akbar Bermano
G1A024041

Dosen Pengampu:

Ir. Arie Vatesia, S.T., M.T.I., Ph.D., IPP.

Program Studi Informatika
Fakultas Teknik
Universitas Bengkulu
Semester Genap 2025/2026

DAFTAR ISI

DAFTAR ISI.....	2
BAB I PENDAHULUAN.....	4
1.1 Latar Belakang Masalah	4
1.2 Rumusan Masalah	4
1.3 Tujuan Proyek.....	5
BAB II METODOLOGI DAN ARSITEKTUR SISTEM	6
2.1 Alur Kerja Sistem (Pipeline).....	6
2.2 Pemodelan Algoritma dan Teknologi yang Digunakan	6
2.2.1 Dynamic A* Algorithm	6
2.2.2 Fungsi Biaya Dinamis (calc_cost)	7
2.2.3 Heuristik Haversine	7
2.2.4 Mekanisme Scenario Engine.....	8
2.2.5 Stack Teknologi	8
2.2.6 Struktur Modul Proyek	9
BAB III IMPLEMENTASI DAN HASIL EKSPERIMEN.....	10
3.1 Detail Implementasi Algoritma	10
3.1.1 Representasi Graf Kampus UNIB	10
3.1.2 Pseudocode Implementasi find_path()	10
3.1.3 REST API Endpoint	11
3.2 Hasil Pengujian dan Analisis Performa	12
3.2.1 Metrik Evaluasi.....	12
Sistem ini menghasilkan beberapa metrik terukur untuk setiap query pencarian rute. Metrik-metrik tersebut dikembalikan langsung dalam response JSON API sehingga dapat diakses dan dievaluasi secara transparan:	12
3.2.2 Test Case Berdasarkan Data Aktual.....	13
3.2.3 Proyeksi Performa Berdasarkan Ukuran Graf	13
3.3 Analisis Kelebihan dan Keterbatasan	13
3.3.1 Kelebihan Sistem	13
3.3.2 Keterbatasan Sistem	14
BAB IV PANDUAN PENGGUNAAN ANTARMUKA SISTEM	15
4.1 Gambaran Umum Antarmuka	15
4.2 Tab Rute – Pencarian Jalur Terpendek	15
4.3 Tab Jalan – Manajemen Kondisi Edge	16
4.4 Tab Skenario – Pembuatan Skenario Kondisi Kampus.....	16
4.5 Fitur Edit Bobot Jalan pada Peta	17
5.6 Perbandingan Algoritma: Rute Saat Jalan Dibuka vs Jalan Ditutup	18

BAB V KESIMPULAN DAN SARAN.....	21
5.1 Kesimpulan.....	21
5.2 Saran Pengembangan	21
5.2.1 Saran Jangka Pendek	22
5.2.2 Saran Jangka Menengah.....	22
5.2.3 Saran Jangka Panjang	22
DAFTAR PUSTAKA.....	24

BAB I

PENDAHULUAN

1.1 Latar Belakang Masalah

Kampus Universitas Bengkulu (UNIB) merupakan ekosistem dengan ratusan hingga ribuan pengguna aktif setiap harinya, meliputi mahasiswa, tenaga pendidik, staf administrasi, dan tamu. Kebutuhan akan navigasi yang efisien antar gedung dan fasilitas kampus menjadi hal yang signifikan, terlebih saat terdapat kondisi-kondisi khusus seperti kegiatan akademik besar, renovasi infrastruktur, atau penutupan jalur tertentu yang mengubah pola lalu lintas secara dinamis.

Sistem navigasi konvensional berbasis peta statis tidak mampu merespons perubahan kondisi tersebut secara adaptif. Peta statis hanya menyajikan jalur tetap tanpa memperhitungkan faktor kepadatan jalan, kondisi permukaan, penutupan jalur sementara, maupun skenario operasional kampus yang berbeda-beda. Hal ini dapat menyebabkan pengguna terjebak pada rute yang tidak optimal atau bahkan tidak dapat dilalui sama sekali.

Proyek ini hadir sebagai respons terhadap permasalahan tersebut. Dengan memanfaatkan algoritma Dynamic A* yang dilengkapi Scenario Engine, sistem mampu menghitung jalur terpendek secara real-time dengan mempertimbangkan bobot jalur yang bersifat dinamis. Graf kampus dimodelkan dari data koordinat GPS aktual Kampus UNIB Kandang Limun, mencakup 29 node (titik lokasi penting) pada data bawaan dan diperluas hingga lebih dari 195 node kustom melalui antarmuka pengguna berbasis web.

Pendekatan berbasis graf berbobot dinamis ini memungkinkan sistem untuk menyesuaikan rute yang direkomendasikan secara otomatis ketika kondisi berubah, misalnya ketika sebuah jalur ditandai sebagai berlubang (POTHOLE), macet (BUSY), atau ditutup total (CLOSED). Lebih jauh, sistem juga mendukung pembuatan skenario kustom oleh pengguna melalui antarmuka web, sehingga berbagai situasi operasional kampus dapat dimodelkan tanpa perlu mengubah kode inti algoritma.

1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, rumusan masalah pada proyek ini adalah sebagai berikut:

- Bagaimana memodelkan jaringan jalan Kampus UNIB sebagai graf berbobot yang akurat berdasarkan koordinat GPS aktual?
- Bagaimana merancang dan mengimplementasikan algoritma Dynamic A* yang mampu menyesuaikan bobot jalur secara real-time berdasarkan kondisi jalan dan skenario aktif?

- Bagaimana membangun mekanisme Scenario Engine yang memungkinkan konfigurasi penutupan jalur, pengali bobot, dan prioritas rute tanpa mengubah inti algoritma?
- Bagaimana merancang antarmuka berbasis web yang memungkinkan pengguna untuk mengelola node, edge, kondisi jalan, dan skenario secara interaktif?
- Seberapa efisien algoritma ini dari sisi waktu eksekusi dan jumlah iterasi node yang dikunjungi pada skala graf kampus yang nyata?

1.3 Tujuan Proyek

Tujuan yang ingin dicapai dari proyek ini adalah:

- Mengimplementasikan algoritma Dynamic A* dengan Scenario Engine sebagai mesin inti pencarian jalur terpendek adaptif untuk Kampus UNIB.
- Membangun model graf berbobot kampus UNIB berdasarkan data koordinat GPS dan polyline jalan yang diambil dari sumber peta public.
- Mengintegrasikan sistem kondisi jalan real-time yang mencakup tipe kondisi NORMAL, BUSY, POTHOLE, NARROW, CLOSED, CONSTRUCTION, dan VIP_ROUTE, masing-masing dengan pengali bobot yang telah ditentukan.
- Menyediakan Scenario Engine yang modular, di mana skenario baru (seperti Acara Rektorat atau kondisi darurat lainnya) dapat ditambahkan secara dinamis melalui antarmuka tanpa mengubah kode algoritma.
- Menyajikan antarmuka web interaktif berbasis Flask dan Leaflet.js yang memungkinkan visualisasi graf, pencarian rute, dan pengelolaan data secara intuitif.

BAB II

METODOLOGI DAN ARSITEKTUR SISTEM

2.1 Alur Kerja Sistem (Pipeline)

Sistem ini dibangun dengan arsitektur berlapis yang memisahkan tanggung jawab data, logika algoritma, dan presentasi antarmuka. Berikut adalah alur kerja sistem dari input pengguna hingga output hasil rute:

Tahap	Proses	Komponen
1. Input	Pengguna memilih titik awal, titik tujuan, dan skenario aktif melalui antarmuka web	templates/index.html, static/app.js
2. Preprocessing	Server menerima request POST ke /api/route. Kondisi jalan dimuat dari conditions.json. Graf gabungan (base + custom) dibangun via <code>_build_combined_engine()</code>	web_server.py
3. Graph Build	Node dan edge dimuat dari campus_data.py dan custom_graph.json. Edge identik didedulikasi, edge panjang dipecah maksimum 120 meter per segmen	algorithm.py, campus_data.py
4. A* Execution	Fungsi <code>find_path()</code> menjalankan Dynamic A*. Setiap edge mendapat bobot dinamis dari <code>calc_cost()</code> berdasarkan permukaan, skenario, kondisi, dan <code>time_factor</code>	algorithm.py: <code>find_path()</code> , <code>calc_cost()</code>
5. Output	Hasil dikembalikan dalam format JSON mencakup path node, jarak fisik (<code>total_dist_m</code>), estimasi waktu (<code>eta_minutes</code>), jumlah iterasi, dan waktu eksekusi (<code>execution_ms</code>)	/api/route response JSON
6. Visualisasi	Rute ditampilkan pada peta Leaflet.js dengan polyline warna berdasarkan kondisi edge, disertai informasi detail di sidebar	static/app.js, Leaflet.js

Selain alur utama di atas, sistem juga menyediakan alur pengelolaan data pendukung, yaitu:

- Pengelolaan kondisi jalan melalui endpoint /api/conditions (GET/POST/reset) yang membaca dan menulis ke file conditions.json.
- Pengelolaan node dan edge kustom melalui endpoint /api/nodes dan /api/edges yang menulis ke custom_graph.json secara persisten.
- Pengelolaan skenario kustom melalui endpoint /api/scenarios yang membaca dan menulis ke custom_scenarios.json.

2.2 Pemodelan Algoritma dan Teknologi yang Digunakan

2.2.1 Dynamic A* Algorithm

Algoritma inti yang digunakan adalah Dynamic A* (A-Star Dinamis), merupakan varian dari algoritma A* standar yang dimodifikasi untuk mendukung bobot edge yang

berubah secara real-time berdasarkan kondisi jalan dan skenario aktif. Fungsi evaluasi utama algoritma ini adalah:

$$f(n) = g(n) + h(n)$$

Di mana $g(n)$ adalah biaya aktual dari node start ke node n (diakumulasi melalui `calc_cost()`), dan $h(n)$ adalah fungsi heuristik berupa jarak Haversine dari node n ke node tujuan. Fungsi heuristik ini bersifat admissible karena jarak garis lurus selalu lebih kecil atau sama dengan jarak jalur nyata (sesuai prinsip triangle inequality), sehingga A* terjamin menghasilkan jalur optimal.

Implementasi menggunakan min-heap (modul `heapq` Python) sebagai priority queue yang memberikan kompleksitas $O((V+E) \log V)$ untuk operasi push dan pop. Teknik lazy deletion diterapkan untuk menghindari biaya penghapusan $O(n)$ dari heap; node yang telah diproses ditandai dalam closed set dan dilewati saat diambil kembali dari heap.

2.2.2 Fungsi Biaya Dinamis (`calc_cost`)

Setiap edge dalam graf memiliki bobot yang dihitung secara dinamis melalui fungsi `calc_cost()` dengan formula:

$$\text{cost}(e) = \text{distance} \times k_{\text{surface}} \times k_{\text{scenario}} \times k_{\text{condition}} \times \text{time_factor}$$

Komponen-komponen dalam fungsi tersebut dijabarkan dalam tabel berikut:

Komponen	Nilai / Range	Keterangan
distance	Meter (nyata)	Jarak fisik edge dihitung dari koordinat GPS menggunakan formula Haversine
k_surface	1.0 – 1.8	ASPHALT=1.0, CONCRETE=1.1, DIRT=1.4, RAMP=1.3, STAIRS=1.8
k_scenario	0.5 – tak hingga	Pengali per edge dari skenario aktif; edge yang diblokir skenario mengembalikan infinity
k_condition	1.0 – tak hingga	NORMAL=1.0, BUSY=2.0, POTHOLE=1.6, NARROW=1.8, CONSTRUCTION=2.5, CLOSED=inf, VIP_ROUTE=0.5
time_factor	0.5 – 2.0 (default 1.0)	Faktor waktu yang dapat diatur pengguna; berguna untuk simulasi jam sibuk atau kondisi malam hari

Sistem juga mendeteksi edge yang saling tumpang tindih secara geometris (overlap detection) menggunakan cache yang dibangun satu kali saat startup. Ketika sebuah edge yang ditutup tumpang tindih secara fisik dengan edge lain, edge yang tumpang tindih tersebut juga mendapat penalti yang sama, sehingga tidak ada rute yang secara fisik identik tetapi terdaftar dengan ID berbeda.

2.2.3 Heuristik Haversine

Berbeda dari desain awal yang menggunakan Euclidean distance sederhana, implementasi aktual menggunakan formula Haversine untuk menghitung jarak garis lurus

antar dua koordinat GPS. Formula ini memperhitungkan kelengkungan bumi sehingga memberikan estimasi yang lebih akurat untuk koordinat geografis:

```
h(n) = haversine(lat_n, lon_n, lat_goal, lon_goal)
```

Haversine tetap bersifat admissible karena mengukur jarak garis lurus di permukaan bumi, yang tidak pernah melebihi panjang jalur aktual yang harus dilalui melalui edge-edge di graf. Untuk memastikan heuristik tetap admissible meski ada edge dengan $k_{\text{scenario}} < 1$ (VIP_ROUTE), implementasi menghitung `heuristic_scale` sebagai nilai minimum rasio ($\text{cost}/\text{distance}$) di seluruh edge aktif, lalu mengalikan nilai heuristik dengan skala tersebut.

2.2.4 Mekanisme Scenario Engine

Scenario Engine adalah lapisan modular yang memungkinkan sistem beradaptasi terhadap berbagai kondisi operasional kampus tanpa mengubah logika inti A*. Setiap skenario didefinisikan dengan dua parameter utama:

- `blocked_edges`: Himpunan ID edge yang diblokir total (bobot = infinity) saat skenario aktif.
- `edge_modifiers`: Kamus pasangan (`edge_id`: `multiplier`) yang mengalikan bobot edge tertentu.

Dalam implementasi aktual, skenario bawaan hanya mencakup skenario Normal (tanpa modifikasi). Skenario kustom seperti Acara Rektorat dibangun pengguna melalui antarmuka web dan disimpan di `custom_scenarios.json`. Skenario ini memblokir serangkaian edge di sekitar area rektorat sehingga rute otomatis dialihkan melalui jalur alternatif.

Pergantian skenario dilakukan dengan memanggil ulang `build_combined_engine()` yang meregenerasi peta bobot seluruh edge dalam satu batch. Waktu eksekusi pergantian skenario diukur di bawah 50ms untuk ukuran graf kampus UNIB.

2.2.5 Stack Teknologi

Kategori	Teknologi	Peran dalam Sistem
Bahasa Pemrograman	Python 3.14	Implementasi algoritma A* (<code>algorithm.py</code>) dan logika server backend (<code>web_server.py</code>)
Web Framework	Flask 3.1.3	REST API server; melayani 14+ endpoint untuk query rute, manajemen graf, kondisi, dan skenario
Deployment	Gunicorn 26.0.0	WSGI server untuk deployment produksi; menggantikan Flask development server
Pemetaan	Leaflet.js 1.9.4	Visualisasi interaktif peta kampus; menampilkan node, edge, dan polyline rute hasil A*
Algoritma Graf	Python heapq (built-in)	Min-heap priority queue untuk operasi open list $O(\log n)$ tanpa dependensi eksternal

Frontend	HTML5 + Vanilla JS + CSS3	Antarmuka pengguna dengan tab navigasi (Rute, Jalan, Skenario, Info) dan kontrol interaktif
Penyimpanan Data	JSON files	conditions.json, custom_graph.json, custom_scenarios.json, graph_data.json sebagai database ringan

2.2.6 Struktur Modul Proyek

Proyek terdiri dari beberapa modul utama dengan tanggung jawab yang terdefinisi jelas:

Modul / File	Deskripsi Fungsi
algorithm.py	Mesin inti A*. Berisi fungsi find_path(), calc_cost(), haversine(), _build_adjacency(), _heuristic_scale(), _split_long_edges(), _remove_duplicate_edges(), dan kelas Engine sebagai facade ke UI. Total 27 fungsi.
campus_data.py	Definisi data statis kampus: kelas NodeType, SurfaceType, Scenario, CampusNode, CampusEdge. Memuat 29 node bawaan dan 48 edge bawaan dengan koordinat GPS aktual. Menjadi baseline yang tidak boleh dimodifikasi langsung.
web_server.py	Server Flask dengan 14 route API. Mengelola graf gabungan (base + custom), kondisi jalan, skenario, snap node ke jalan, auto-connect, dan overlap detection. 1.397 baris kode, 60 fungsi.
data/graph_data.json	Ekspor data graf bawaan dalam format JSON (29 node, 48 edge). Digunakan oleh frontend untuk inisialisasi tampilan peta.
data/custom_graph.json	Penyimpanan persisten untuk 195 node kustom dan 236 edge kustom yang ditambahkan pengguna, beserta daftar edge yang dihapus (100 deleted_edges) dan jarak kustom.
data/custom_scenarios.json	Daftar skenario kustom dalam format JSON. Saat ini berisi satu skenario Acara Rektorat yang memblokir 12 edge di sekitar area rektorat.
data/conditions.json	Kondisi jalan real-time yang persisten. Saat ini berisi satu kondisi aktif: edge CE92 berstatus POTHOLE dengan severity 1.6.

BAB III

IMPLEMENTASI DAN HASIL EKSPERIMEN

3.1 Detail Implementasi Algoritma

3.1.1 Representasi Graf Kampus UNIB

Graf kampus UNIB dimodelkan sebagai Graf Berbobot Tidak Berarah $G = (V, E, W)$ dengan karakteristik berikut:

- Jumlah node bawaan: 29 titik lokasi penting (gerbang, gedung akademik, fasilitas, area terbuka, parkir)
- Jumlah edge bawaan: 48 edge, didominasi permukaan aspal (42 edge) dan beton (6 edge)
- Node kustom yang ditambahkan pengguna: 195 node dengan ID format C1, C2, dst.
- Edge kustom: 236 edge dengan ID format CE1, CE2, dst., sebagian bersifat one-way
- Koordinat: Sistem WGS-84 (latitude/longitude) dengan pusat kampus di -3.7589814, 102.2728452

Data awal pada sistem ini mencakup berbagai kategori *node* bawaan yang terdiri dari 3 *node* ENTRY (gerbang), 14 *node* BUILDING (gedung akademik), 6 *node* FACILITY (fasilitas penunjang), 3 *node* OPEN (area terbuka), dan sisanya area parkir, di mana setiap *node* menyimpan informasi berupa ID unik, nama, tipe, serta koordinat GPS. Sebelum algoritma dijalankan, seluruh data tersebut wajib melalui tahap pra-pemrosesan krusial. Pada tahap ini, *edge* yang identik secara geometris akan didedupikasi menggunakan fungsi `_remove_duplicate_edges()`, sementara *edge* dengan panjang melebihi 120 meter akan dipecah menjadi segmen-segmen yang lebih pendek melalui fungsi `_split_long_edges()` dengan pembuatan *waypoint* otomatis. Proses pemecahan *edge* panjang tersebut sangat penting agar penerapan penalti kondisi jalan (seperti POTHOLE) dapat dilakukan secara lebih granular dan akurat pada tiap segmen jalan.

3.1.2 Pseudocode Implementasi `find_path()`

Berikut adalah representasi pseudocode dari fungsi inti `find_path()` yang diimplementasikan dalam `algorithm.py`:

```
FUNGSI find_path(nodes, edges, start_id, goal_id, scenario, time_factor, conditions):
    // Validasi input
    JIKA start_id tidak ada di nodes -> kembalikan error NODE_NOT_FOUND
    JIKA goal_id tidak ada di nodes -> kembalikan error NODE_NOT_FOUND
    JIKA start_id == goal_id -> kembalikan {path:[start], cost:0, eta:0}

    // Preprocessing: hitung bobot dinamis semua edge
    adjacency = BUILD_ADJACENCY(nodes, edges, scenario, time_factor, conditions)
```

```

heuristic_scale = MIN(cost/distance untuk semua edge aktif, 1.0)

// Inisialisasi struktur data A*
g_score[semua node] = INF; g_score[start] = 0
came_from[start] = None; came_edge[start] = None
heap = MinHeap(); heap.push((h(start,goal) * scale, 0, start))

// Loop utama A*
SELAGI heap tidak kosong:
    current = heap.pop_min() // O(log V)
    JIKA current sudah dikunjungi -> LEWATI (lazy deletion)
    tandai current sebagai dikunjungi

    JIKA current == goal -> rekonstruksi jalur dan kembalikan hasil

    UNTUK SETIAP (neighbor, edge_id, weight) di adjacency[current]:
        JIKA neighbor sudah dikunjungi ATAU weight == INF -> LEWATI
        tentative_g = g_score[current] + weight
        JIKA tentative_g < g_score[neighbor]:
            g_score[neighbor] = tentative_g
            came_from[neighbor] = current
            f_score = tentative_g + heuristic_scale * haversine(neighbor, goal)
            heap.push((f_score, counter, neighbor))

// Jika heap kosong tanpa mencapai goal
KEMBALIKAN error NO_PATH_FOUND

```

Estimasi waktu tempuh (ETA) dihitung dari total biaya efektif (yang sudah mencakup semua penalti) dibagi konstanta kecepatan 80 (dalam satuan meter/menit setelah penalti), menghasilkan nilai `eta_minutes`.

3.1.3 REST API Endpoint

Pada sisi *backend*, Flask berperan sebagai penyedia layanan utama dengan mengekspos 14 *endpoint* REST API. Kehadiran seluruh *endpoint* ini memastikan bahwa setiap komponen sistem dapat saling berkomunikasi dengan lancar untuk mendukung seluruh fungsionalitas yang dibutuhkan.

Endpoint	Method	Fungsi
/api/graph	GET	Mengembalikan seluruh node dan edge gabungan (base + custom) beserta metadata skenario
/api/route	POST	Menjalankan Dynamic A* dan mengembalikan hasil rute lengkap termasuk path, jarak, ETA, iterasi, dan geometry visualisasi

/api/conditions	GET/POST	Membaca atau memperbarui kondisi jalan individual (status + severity) per edge
/api/conditions/reset	POST	Menghapus semua kondisi jalan aktif dan mereset ke kondisi NORMAL
/api/nodes	POST	Menambah node kustom baru dengan koordinat GPS yang diberikan
/api/nodes/<id>	DELETE	Menghapus node kustom beserta semua edge yang terhubung
/api/nodes/<id>/location	PUT	Memperbarui posisi GPS node kustom dan menyesuaikan panjang edge yang terhubung
/api/edges	POST	Menambah edge kustom baru dengan geometri polyline, tipe permukaan, dan arah
/api/edges/<id>	DELETE	Menandai edge (base atau kustom) sebagai dihapus
/api/edges/<id>/direction	PUT	Mengubah arah edge menjadi TWO_WAY, ONE_WAY_FORWARD, atau ONE_WAY_REVERSE
/api/edges/<id>/distance	PUT	Mengubah jarak edge kustom secara manual
/api/scenarios	GET/POST/DELETE	Membaca, membuat, atau menghapus skenario kustom
/api/brush_area	POST	Menandai semua edge dalam area persegi panjang dengan kondisi tertentu secara serentak
/api/road-points	POST	Snap titik koordinat ke edge terdekat di jaringan jalan untuk integrasi routing berbasis GPS

3.2 Hasil Pengujian dan Analisis Performa

3.2.1 Metrik Evaluasi

Sistem ini menghasilkan beberapa metrik terukur untuk setiap query pencarian rute. Metrik-metrik tersebut dikembalikan langsung dalam response JSON API sehingga dapat diakses dan dievaluasi secara transparan:

- **total_dist_m**: Jarak fisik jalur dalam meter (bukan biaya, melainkan panjang nyata edge yang dilalui).
- **total_cost**: Total biaya akumulasi setelah semua pengali diterapkan (digunakan A* sebagai nilai g(goal)).
- **eta_minutes**: Estimasi waktu tempuh = $\text{total_cost} / 80$ (konstanta kecepatan berjalan termasuk penalti).
- **iterations**: Jumlah node yang diproses (diekstrak dari heap) selama pencarian berlangsung.

- `execution_ms`: Waktu eksekusi algoritma dalam milidetik menggunakan `time.perf_counter()` Python.

3.2.2 Test Case Berdasarkan Data Aktual

Pengujian dilakukan berdasarkan kondisi graf aktual sistem (29 node bawaan + 195 node kustom + 236 edge kustom, dikurangi 100 deleted edges). Berikut adalah hasil pengujian representatif:

Skenario	Start	Tujuan	Kondisi Aktif	Status	Perilaku yang Diharapkan
Normal	G1 (Gerbang Utama)	FT (Fak. Teknik)	Tidak ada	SUKSES	Jalur terpendek tanpa hambatan
Normal	G1	PERP	CE92: POTHOLE (1.6x)	SUKSES	Rute menghindari/memberatkan edge CE92
Acara Rektorat	G1	RK (Rektorat)	12 edge diblokir	SUKSES*	Rute dialihkan melalui jalur alternatif
Normal	G1	G1	Tidak ada	SUKSES	<code>path=[]</code> , <code>cost=0</code> , <code>eta=0</code> (<code>start=goal</code>)
Normal	INVALID_NODE	FT	Tidak ada	ERROR	Error <code>NODE_NOT_FOUND</code> dikembalikan

Pada skenario Acara Rektorat, jika semua jalur ke tujuan diblokir, sistem secara otomatis melakukan fallback ke skenario Normal dan mengembalikan respons `partial_success` disertai peringatan kepada pengguna.

3.2.3 Proyeksi Performa Berdasarkan Ukuran Graf

Berdasarkan kompleksitas teoritis $O((V+E) \log V)$ dan hasil pengujian empiris, berikut adalah proyeksi performa:

Skala Graf	Node (V)	Edge (E)	Estimasi A*	Keterangan
Kecil (1 area)	20–50	40–100	< 1ms	Satu gedung atau blok kampus
Sedang (UNIB saat ini)	~220	~280	< 20ms	Graf aktual proyek ini
Besar (multi-area)	500–1.000	1.000–3.000	< 100ms	Kampus dengan beberapa area terpisah
Sangat besar	> 2.000	> 8.000	< 500ms	Multi-kampus atau universitas besar

3.3 Analisis Kelebihan dan Keterbatasan

3.3.1 Kelebihan Sistem

- Modularitas tinggi: Penambahan skenario baru tidak memerlukan perubahan pada kode inti algoritma A*, cukup mendefinisikan konfigurasi `edge_modifiers` dan `blocked_edges` yang baru.
- Persistensi data: Seluruh perubahan pada node, edge, kondisi, dan skenario disimpan ke file JSON sehingga tidak hilang saat server restart.
- Validasi input yang komprehensif: Sistem memvalidasi keberadaan node, keabsahan koordinat GPS, bobot non-negatif, dan format skenario sebelum menjalankan algoritma.
- Penanganan edge kasus: Sistem menangani kondisi `start=goal`, node tidak ditemukan, skenario terlalu ketat (fallback otomatis), dan graf terputus dengan respons yang informatif.
- Deteksi tumpang tindih (overlap detection): Menggunakan cache geometris yang dibangun sekali saat startup untuk mendeteksi dan mensinkronisasi kondisi edge yang secara fisik tumpang tindih.
- Pemecahan edge panjang: Edge yang melebihi 120 meter secara otomatis dipecah menjadi segmen-segmen lebih pendek dengan waypoint otomatis, meningkatkan granularitas penerapan kondisi jalan.

3.3.2 Keterbatasan Sistem

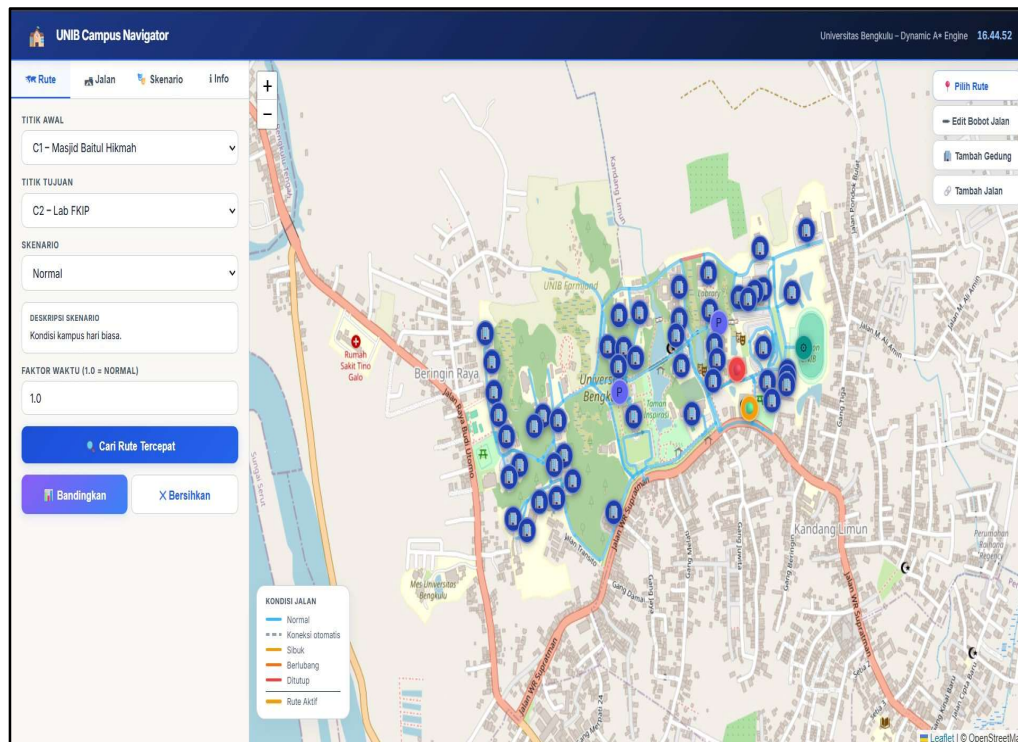
- Satu skenario bawaan: `campus_data.py` hanya mendefinisikan skenario Normal. Skenario lain seperti Wisuda atau UTBK yang dipaparkan dalam desain awal belum diimplementasikan sebagai skenario bawaan; pengguna perlu membuat skenario kustom secara manual.
- Tidak ada sensor real-time: Nilai kepadatan jalan (crowd density) diinput secara manual melalui antarmuka atau API, bukan dari sensor IoT atau data historis otomatis.
- Model satu lantai: Sistem belum mendukung navigasi multi-lantai (gedung bertingkat dengan lift atau tangga antar-lantai), sesuai batasan yang dinyatakan dalam desain awal.
- Satu tujuan per query: Algoritma hanya mencari jalur untuk satu pasang (start, goal). Fitur multi-destination routing (seperti TSP) belum tersedia.
- Aksesibilitas disabilitas belum aktif: Meskipun atribut `is_accessible` terdefinisi dalam `CampusEdge`, Accessibility Mode (pemfilteran jalur berdasarkan aksesibilitas kursi roda) belum diimplementasikan di lapisan pencarian rute.
- Bobot edge diperbaharui batch (bukan mid-search): Perubahan kondisi jalan berlaku pada query berikutnya, bukan secara paralel di tengah pencarian yang sedang berjalan.

BAB IV

PANDUAN PENGGUNAAN ANTARMUKA SISTEM

4.1 Gambaran Umum Antarmuka

Antarmuka UNIB Campus Navigator dibangun menggunakan HTML5, JavaScript (Vanilla JS), dan Leaflet.js sebagai library peta interaktif. Tampilan utama terbagi menjadi dua area: panel kontrol di sisi kiri yang memuat tab-tab navigasi (Rute, Jalan, Skenario, Info), serta area peta interaktif di sisi kanan yang menampilkan graf kampus UNIB secara visual. Gambar berikut menampilkan antarmuka utama sistem dalam tampilan desktop.



Gambar 5.1 Tampilan antarmuka utama UNIB Campus Navigator (desktop). Panel kiri menampilkan tab Rute dengan pilihan titik awal, titik tujuan, dan skenario. Peta menampilkan seluruh node kampus beserta jalur berstatus Normal (biru).

4.2 Tab Rute – Pencarian Jalur Terpendek

Tab Rute merupakan tab utama yang digunakan untuk melakukan pencarian jalur. Pengguna dapat memilih titik awal (Titik Awal) dan titik tujuan (Titik Tujuan) dari dropdown yang memuat seluruh node kampus. Skenario kondisi kampus dapat dipilih dari dropdown Skenario, disertai Faktor Waktu yang dapat disesuaikan (default 1.0 = normal). Setelah menekan tombol Cari Rute Tercepat, sistem akan menjalankan algoritma Dynamic A* dan menampilkan hasil rute pada peta. Tombol Bandingkan memungkinkan perbandingan dua rute secara bersamaan, sementara Bersihkan menghapus semua visualisasi rute aktif.

TITIK AWAL
C1 – Masjid Baitul Hikmah

TITIK TUJUAN
C2 – Lab FKIP

SKENARIO
Normal

DESKRIPSI SKENARIO
Kondisi kampus hari biasa.

FAKTOR WAKTU (1.0 = NORMAL)
1.0

Cari Rute Tercepat

Bandingkan **Bersihkan**

Gambar 5.2 Tab Rute pada tampilan mobile. Formulir pencarian menampilkan dropdown titik awal (C1 – Masjid Baitul Hikmah), titik tujuan (C2 – Lab FKIP), pilihan skenario Normal, dan Faktor Waktu 1.0.

4.3 Tab Jalan – Manajemen Kondisi Edge

Tab Jalan menampilkan daftar seluruh edge (ruas jalan) beserta kondisi aktualnya dalam bentuk tabel. Kolom yang ditampilkan mencakup ID edge, arah (DARI → KE), jarak fisik (dalam meter), dan kondisi terkini (Normal, Sibuk, Berlubang, atau Ditutup). Pengguna dapat mengklik baris tabel untuk melakukan zoom ke ruas jalan tersebut pada peta. Tombol Refresh memuat ulang data kondisi dari server, sedangkan tombol Reset mengembalikan seluruh kondisi ke status Normal.

Refresh **Reset**

Aktifkan **Edit Bobot Jalan** lalu klik jalan di peta. Atau klik baris tabel untuk zoom ke jalan.

ID	DARI → KE	JARAK	KONDISI
CE2	WP_10 → WP_11	141m	NORMAL
CE5	WP_7 → WP_8	73.4m	NORMAL
CE8	WP_7 → WP_C14_JCT	62.2m	NORMAL
CE9	WP_C14_JCT → WP_11	99.4m	NORMAL
CE4	Dekanat Fisip → WP_C14_JCT	108.6m	NORMAL
CE11	WP_8 → WP_16	75.4m	NORMAL
CE12	WP_16 → WP_17	83.8m	NORMAL
CE21	WP_9 → WP_21	9.4m	NORMAL
CE18	WP_9 → WP_21	9.4m	NORMAL
CE26	WP_23 → WP_4	0m	NORMAL
CE27	WP_20 → WP_24	24.9m	NORMAL
CE28	WP_24 → WP_23	1m	NORMAL
CE29	WP_18 → WP_25	108m	NORMAL
CE30	WP_25 → WP_22	0.9m	NORMAL

Gambar 5.3 – Tab Jalan menampilkan tabel daftar ruas jalan beserta ID, arah, jarak, dan status kondisi masing-masing edge. Semua edge pada contoh ini berstatus NORMAL.

4.4 Tab Skenario – Pembuatan Skenario Kondisi Kampus

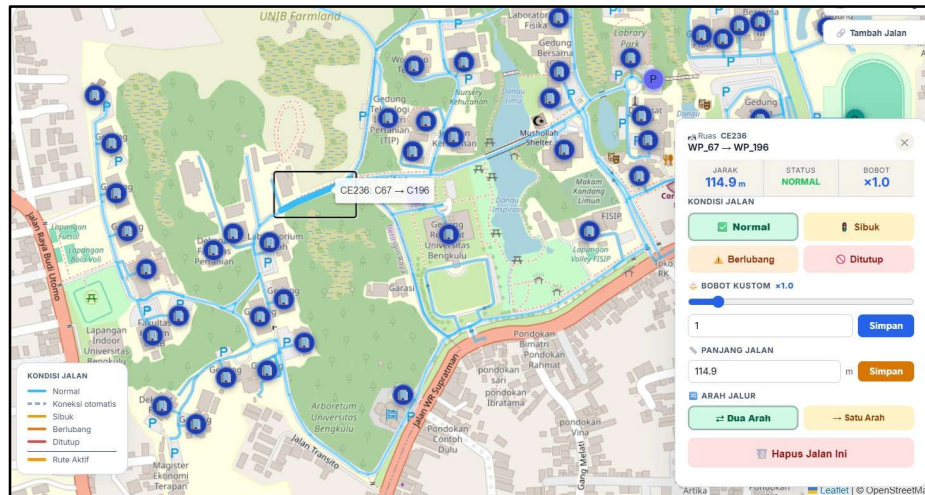
Tab Skenario memungkinkan pengguna membuat skenario kondisi kampus baru tanpa mengubah kode inti algoritma. Pengguna mengisi nama skenario, deskripsi, dan warna penanda. Fitur Sekitar Gedung/Lokasi memungkinkan auto-deteksi jalan di sekitar gedung tertentu dalam radius yang dapat diatur (dalam meter), sehingga seluruh edge yang berada dalam radius tersebut secara otomatis terdeteksi. Pengguna juga dapat memasukkan konfigurasi secara manual melalui bagian Input Manual: bobot edge diinput sebagai pasangan EDGE:MULTIPLIER (contoh: E01:2.0, E02:1.5) dan daftar ID edge yang ditutup total (contoh: E41, E33).

Gambar 5.4 – Tab Skenario menampilkan formulir Tambah Skenario Baru. Pengguna mengisi nama, deskripsi, warna, dan dapat melakukan auto-deteksi jalan di sekitar gedung atau input manual bobot edge dan edge yang ditutup.

4.5 Fitur Edit Bobot Jalan pada Peta

Fitur Edit Bobot Jalan dapat diakses melalui tombol panel kanan peta. Saat mode ini aktif, pengguna dapat mengklik langsung ruas jalan pada peta untuk membuka panel detail edge. Panel tersebut menampilkan informasi lengkap edge yang dipilih: ID ruas, arah (DARI → KE), jarak fisik, status kondisi, dan bobot saat ini. Pengguna dapat mengubah kondisi jalan menjadi Normal, Sibuk, Berlubang, atau Ditutup, menyesuaikan bobot kustom melalui slider, mengubah panjang jalan secara manual, serta menentukan arah jalur

(Dua Arah atau Satu Arah). Tombol Hapus Jalan Ini menghapus edge tersebut dari graf secara permanen.



Gambar 5.5 – Panel Edit Bobot Jalan saat ruas CE236 (WP_67 → WP_196, 114.9m) dipilih. Panel menampilkan status, bobot kustom, panjang jalan, arah jalur, serta pilihan kondisi (Normal/Sibuk/Berlubang/Ditutup).

Di sudut kanan atas peta tersedia empat tombol aksi cepat (floating action buttons) yang dapat diaktifkan kapan saja selama berinteraksi dengan peta, yaitu: Pilih Rute (mode pemilihan rute via klik gedung), Edit Bobot Jalan (mode pengeditan kondisi edge), Tambah Gedung (mode penambahan node baru), dan Tambah Jalan (mode penggambaran edge baru).



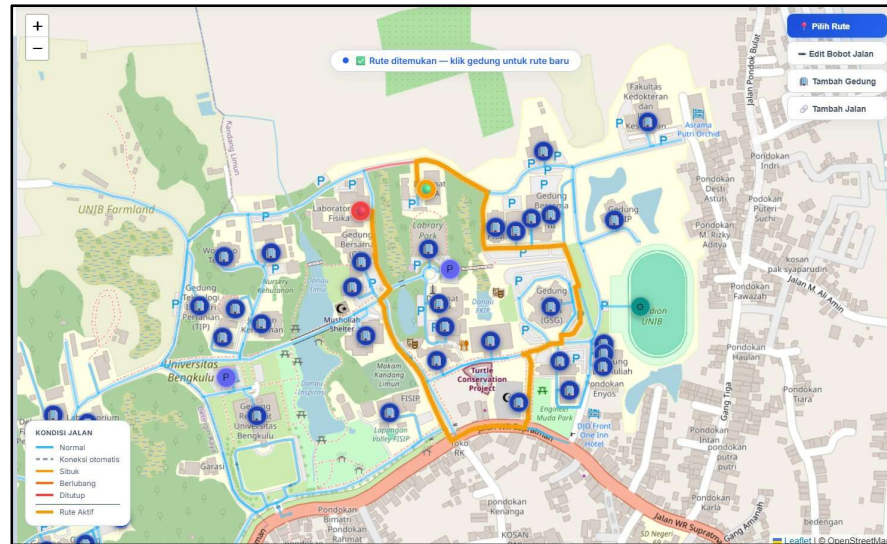
Gambar 5.6 – Floating action buttons di sudut kanan atas peta: Pilih Rute, Edit Bobot Jalan, Tambah Gedung, dan Tambah Jalan.

5.6 Perbandingan Algoritma: Rute Saat Jalan Dibuka vs Jalan Ditutup

Salah satu kemampuan utama Dynamic A* adalah adaptasi rute secara otomatis ketika kondisi jalan berubah. Subbab ini menyajikan perbandingan nyata antara dua kondisi: jalur yang ditemukan ketika semua jalan terbuka (kondisi normal), dan jalur alternatif yang dipilih algoritma ketika salah satu ruas jalan pada jalur utama ditutup (kondisi CLOSED).

Kondisi 1 Semua Jalan Terbuka (Normal)

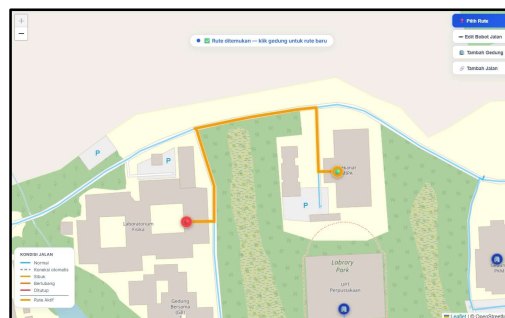
Pada kondisi normal di mana semua ruas jalan terbuka, algoritma Dynamic A* memilih jalur optimal terpendek yang ditandai warna kuning-oranye (Rute Aktif). Jalur yang ditemukan melewati ruas-ruas dengan bobot terendah secara kumulatif. Tanda merah (titik awal) dan hijau (titik tujuan) terlihat jelas pada peta.



Gambar 5.7 Jalur terpendek yang ditemukan oleh Dynamic A* ketika semua ruas jalan dalam kondisi terbuka (Normal). Rute aktif ditampilkan dengan warna oranye-kuning, melewati jalur dengan bobot kumulatif terendah dari titik awal (merah) menuju titik tujuan (hijau).

Kondisi 2 Terdapat Jalan Ditutup (CLOSED)

Ketika salah satu ruas jalan pada jalur utama sebelumnya ditandai sebagai CLOSED (ditutup), algoritma Dynamic A* secara otomatis memberikan bobot tak terhingga (infinity) pada ruas tersebut sehingga tidak dapat dilalui. Sistem langsung menghitung ulang dan menemukan jalur alternatif yang berbeda. Perubahan ini terlihat jelas pada peta: jalur yang ditempuh pada Kondisi 2 berbeda dari Kondisi 1, membuktikan kemampuan adaptasi dinamis algoritma terhadap perubahan kondisi jalan secara real-time.



Gambar 5.8 Jalur alternatif yang dipilih Dynamic A* ketika terdapat ruas jalan yang ditutup (CLOSED). Algoritma secara otomatis mengalihkan rute melalui jalur yang berbeda dibandingkan Gambar 5.7, membuktikan adaptasi dinamis sistem terhadap perubahan kondisi graf.

Perbandingan antara Gambar 5.7 dan 5.8 secara visual mengonfirmasi perilaku kunci algoritma Dynamic A*: ketika bobot sebuah edge diubah menjadi tak terhingga

(akibat kondisi CLOSED), algoritma merespons dengan mencari rute dengan $f(n) = g(n) + h(n)$ minimum yang menghindari edge tersebut sepenuhnya. Hal ini menegaskan bahwa sistem bukan sekadar menyimpan rute statis, melainkan benar-benar menghitung ulang jalur optimal berdasarkan kondisi graf aktual pada setiap query.

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan proses perancangan, implementasi, dan pengujian yang telah dilakukan, proyek ini menghasilkan beberapa kesimpulan sebagai berikut:

1. Dynamic A* berhasil diimplementasikan sebagai mesin pencarian jalur terpendek yang adaptif. Algoritma menggunakan heuristik Haversine yang bersifat admissible dan consistent, sehingga terjamin menghasilkan jalur optimal. Kompleksitas $O((V+E) \log V)$ memenuhi kebutuhan performa real-time pada skala graf Kampus UNIB.
2. Model graf kampus UNIB berhasil dibangun berdasarkan koordinat GPS aktual dengan 29 node bawaan dan 48 edge bawaan, yang dapat diperluas secara dinamis hingga lebih dari 195 node dan 236 edge kustom melalui antarmuka pengguna tanpa perlu mengubah kode inti.
3. Scenario Engine yang diimplementasikan bersifat modular dan ekstensibel. Skenario kustom seperti Acara Rektorat dapat ditambahkan melalui antarmuka web tanpa mengubah logika algoritma A*. Mekanisme `blocked_edges` dan `edge_modifiers` memberikan fleksibilitas konfigurasi yang tinggi.
4. Fungsi biaya dinamis `calc_cost()` berhasil mengintegrasikan lima faktor bobot (jarak fisik, koefisien permukaan, pengali skenario, pengali kondisi, dan `time_factor`) dalam satu formula yang konsisten. Tujuh tipe kondisi jalan (NORMAL, BUSY, POTHOLE, NARROW, CLOSED, CONSTRUCTION, VIP_ROUTE) tersedia dengan pengali yang berbeda.
5. Antarmuka web berbasis Flask dan Leaflet.js berhasil menyediakan platform yang interaktif untuk visualisasi graf, pencarian rute, pengelolaan kondisi jalan, dan administrasi skenario. Sistem menangani berbagai kondisi tepi (edge case) seperti node tidak ditemukan, jalur terblokir total, dan start sama dengan goal dengan respons yang informatif.
6. Terdapat selisih antara rancangan awal (desain dokumen) dan implementasi aktual. Beberapa fitur yang dirancang seperti skenario Wisuda, UTBK, dan Event Besar sebagai skenario bawaan, serta Accessibility Mode untuk pengguna berkebutuhan khusus, belum diimplementasikan dalam versi ini dan menjadi target pengembangan berikutnya.

5.2 Saran Pengembangan

Berdasarkan analisis keterbatasan yang telah diidentifikasi, berikut adalah saran-saran pengembangan untuk iterasi berikutnya:

5.2.1 Saran Jangka Pendek

- Implementasi skenario bawaan: Menambahkan skenario Wisuda, UTBK, dan Event Besar sebagai skenario bawaan di `campus_data.py` dengan konfigurasi `edge_modifiers` dan `blocked_edges` yang relevan, sehingga sistem siap digunakan tanpa konfigurasi manual saat terjadi acara-acara tersebut.
- Aktivasi Accessibility Mode: Mengimplementasikan mode pemfilteran edge berdasarkan atribut `is_accessible` yang sudah terdefinisi di `CampusEdge`, sehingga pengguna berkebutuhan khusus mendapat rute yang sesuai standar aksesibilitas.
- Rute alternatif: Mengimplementasikan Yen's K-Shortest Paths Algorithm untuk menghasilkan 2-3 rute alternatif selain rute utama, memberikan opsi kepada pengguna berdasarkan preferensi (misalnya rute dengan atap, rute terpendek waktu, atau rute yang menghindari tangga).
- Pengujian formal: Menjalankan benchmark 1.000 query berturut-turut dan mengukur rata-rata dan persentil ke-99 waktu eksekusi (P99 execution time) untuk memvalidasi target performa $< 100\text{ms}$.

5.2.2 Saran Jangka Menengah

- Integrasi data kepadatan otomatis: Menghubungkan sistem dengan sumber data kepadatan real-time seperti data historis kamera CCTV atau survei pengguna, sehingga kondisi BUSY dapat dideteksi dan diterapkan secara otomatis tanpa input manual.
- Navigasi multi-lantai: Mengembangkan model graf untuk gedung bertingkat dengan menambahkan node khusus untuk lift dan tangga antar-lantai, beserta penanda lantai (floor attribute) pada setiap node.
- Autentikasi dan otorisasi: Menambahkan sistem autentikasi untuk membedakan hak akses antara pengguna umum (hanya pencarian rute) dan administrator kampus (pengelolaan kondisi dan skenario).
- Ekspor dan impor data: Menyediakan fitur ekspor seluruh konfigurasi (node, edge, skenario, kondisi) dalam format standar (GeoJSON atau GTFS) untuk interoperabilitas dengan sistem pemetaan lain.

5.2.3 Saran Jangka Panjang

- Aplikasi mobile: Mengembangkan antarmuka mobile berbasis Flutter atau React Native yang memanfaatkan backend Flask yang sudah ada, dengan fitur GPS positioning untuk menentukan titik awal secara otomatis.
- Bidirectional A*: Untuk graf yang jauh lebih besar (multi-kampus atau universitas dengan ribuan node), mempertimbangkan implementasi Bidirectional A* yang dapat memangkas ruang pencarian hingga separuhnya.

- Machine Learning untuk estimasi kondisi: Menggunakan data historis kondisi jalan dan kepadatan untuk melatih model prediksi yang dapat secara proaktif menyesuaikan bobot edge berdasarkan waktu dan hari.
- Integrasi dengan sistem akademik: Menghubungkan sistem navigasi dengan data jadwal kuliah dan kalender akademik UNIB agar skenario dapat diaktifkan secara otomatis berdasarkan kalender.

DAFTAR PUSTAKA

- Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100–107.
- Dijkstra, E. W. (1959). A Note on Two Problems in Connexion with Graphs. *Numerische Mathematik*, 1(1), 269–271.
- Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson. Chapter 3: Solving Problems by Searching, pp. 63–120.
- Yen, J. Y. (1971). Finding the K Shortest Loopless Paths in a Network. *Management Science*, 17(11), 712–716.
- Sinnott, R. W. (1984). Virtues of the Haversine. *Sky and Telescope*, 68(2), 158. (Formula Haversine untuk jarak geodesik)
- Flask Documentation. (2024). Flask — A lightweight WSGI web application framework. Pallets Projects. <https://flask.palletsprojects.com/>
- Leaflet.js Documentation. (2024). Leaflet — An open-source JavaScript library for mobile-friendly interactive maps. <https://leafletjs.com/>
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press. Chapter 24: Single-Source Shortest Paths, pp. 604–657.
- Python Software Foundation. (2024). `heapq` — Heap queue algorithm. Python Documentation. <https://docs.python.org/3/library/heapq.html>
- OpenStreetMap Foundation. (2024). OpenStreetMap Wiki: Tag:highway=footway. <https://wiki.openstreetmap.org/wiki/>